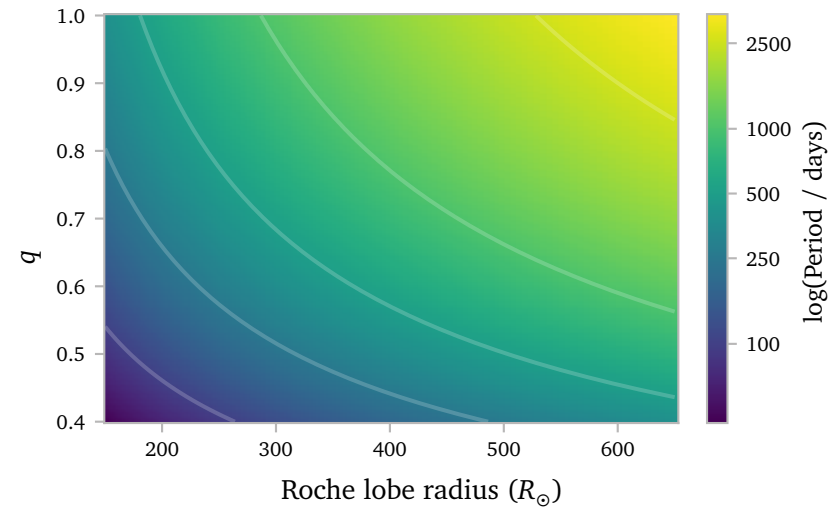


Notes Week 15 + 16

a. Non-conservative Mass Transfer ($\delta = 0.5$)

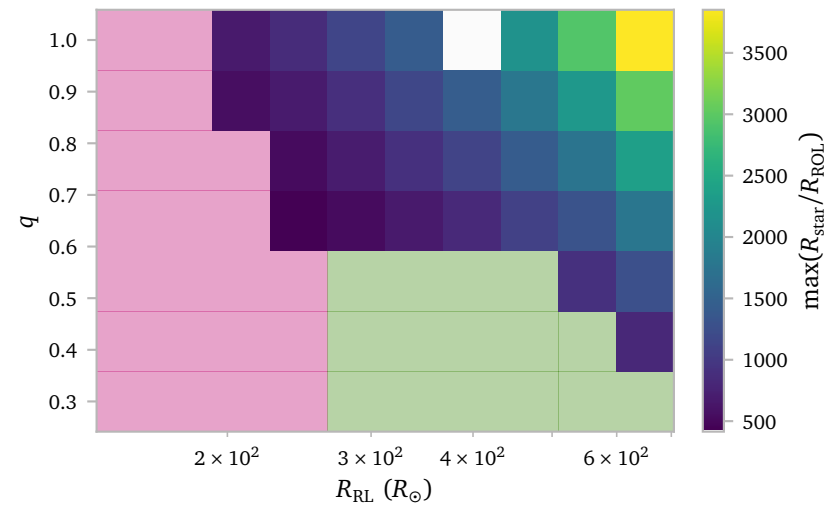
To start, let me see what the analytic expressions from [Soberman et al. \(1997\)](#) predict for the final periods of the our grid.

This is for $\delta = 0.5$ and $\gamma = 1$.



If our MESA models align well with the analytical expression, we should obtain periods towards the lower end of the observed barium star distribution. Of course, the MESA models also have non-conservative wind mass losses, which should alter the results a little bit.

This is the actual grid.



The first thing to notice is that the region of systems that do not finish their evolution has grown substantially, even with respect to the already large region of unstable systems that we saw when we modelled the evolution with $\delta = 0.2$ and $\gamma = 1.25$. See Figures C

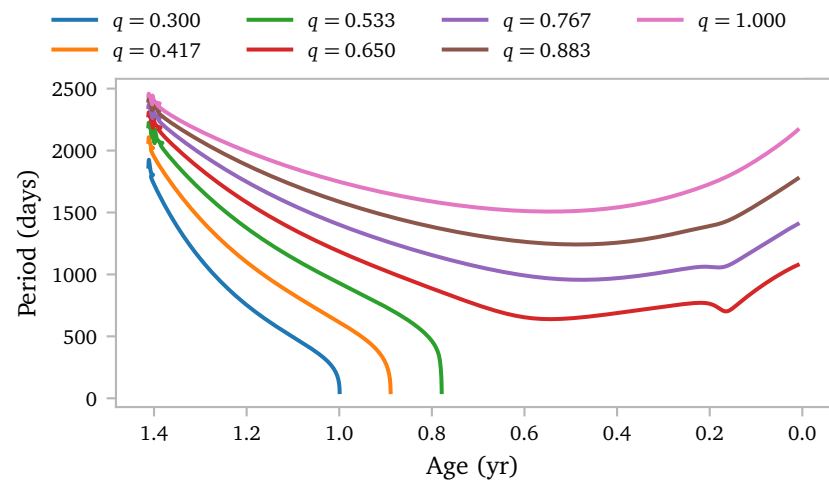
Then, it is also nice to see that we *can* produce these pretty short period systems with *MESA*. The question is obviously: How accurate are these results?

The pink regions are models that failed due to convergence problems. The green regions are models that failed because the binary reached an orbit smaller than 50 days, after which the simulation gets halted.

The single white model had a problem with saving a photo, similar to what we saw for a model in another grid.

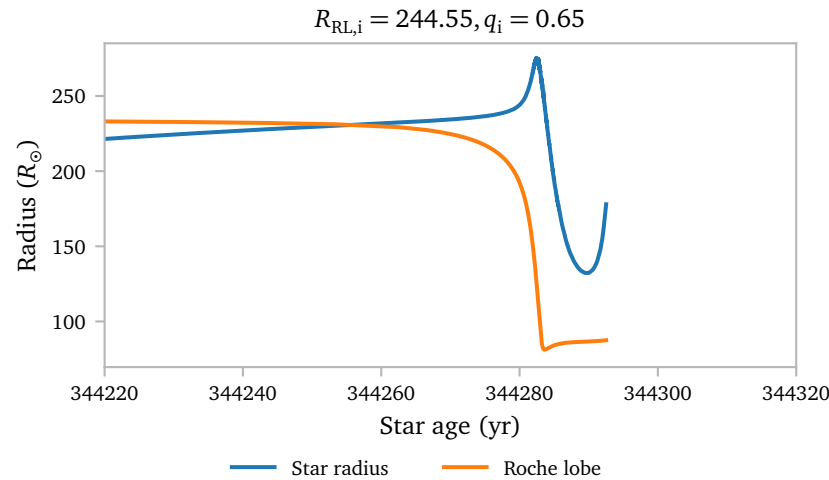
One other thing to note: I accidentally simulated models with slightly different q values, this time from 0.3 to 1 in 7 steps, which is slightly awkward, but not necessarily bad. It does mean one needs to be careful when directly comparing this grid to the previous ones.

Below I plot the period evolution of one of the columns.



I think the blue and orange model reach the Darwin Instability criterion. Is our basic tidal prescription able to accurately predict how tides should behave during RLOF?

Below, I plot the radial evolution of the most extreme model that survives the evolution.



This is really extreme! Also, the time of RLOF is much shorter than in the case of fully conservative mass transfer. Here it only lasts for 30 something years.

b. Different Masses

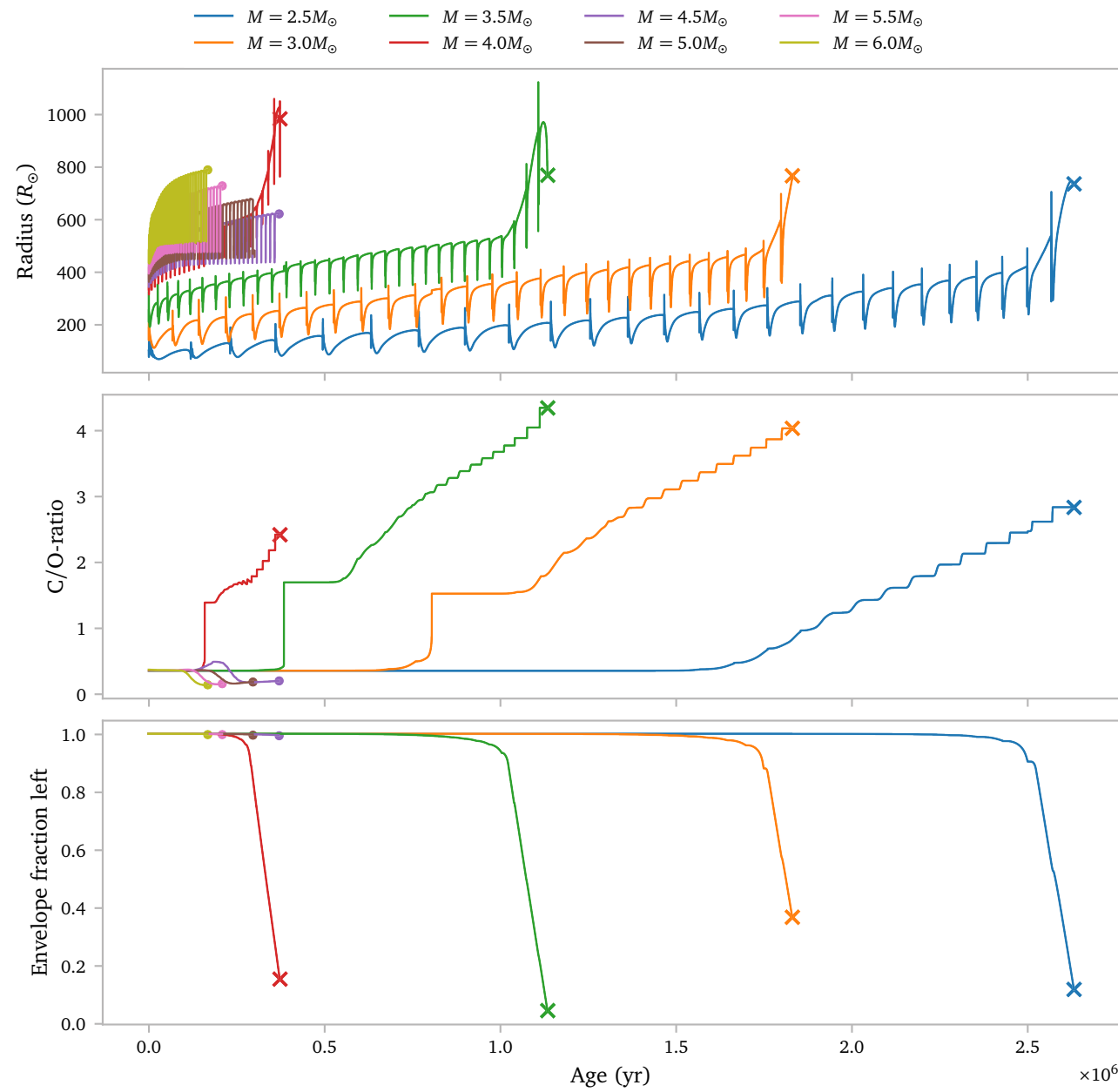
b.1 Model Saving Code

I created yet another version of my model saving code. Now, it saves 1 model per thermal pulse + interpulse cycle with the maximum radius that was reached during this pulse. The model is saved at the start of a new pulse.¹

¹ The code can be seen in appendix A

b.2 Results

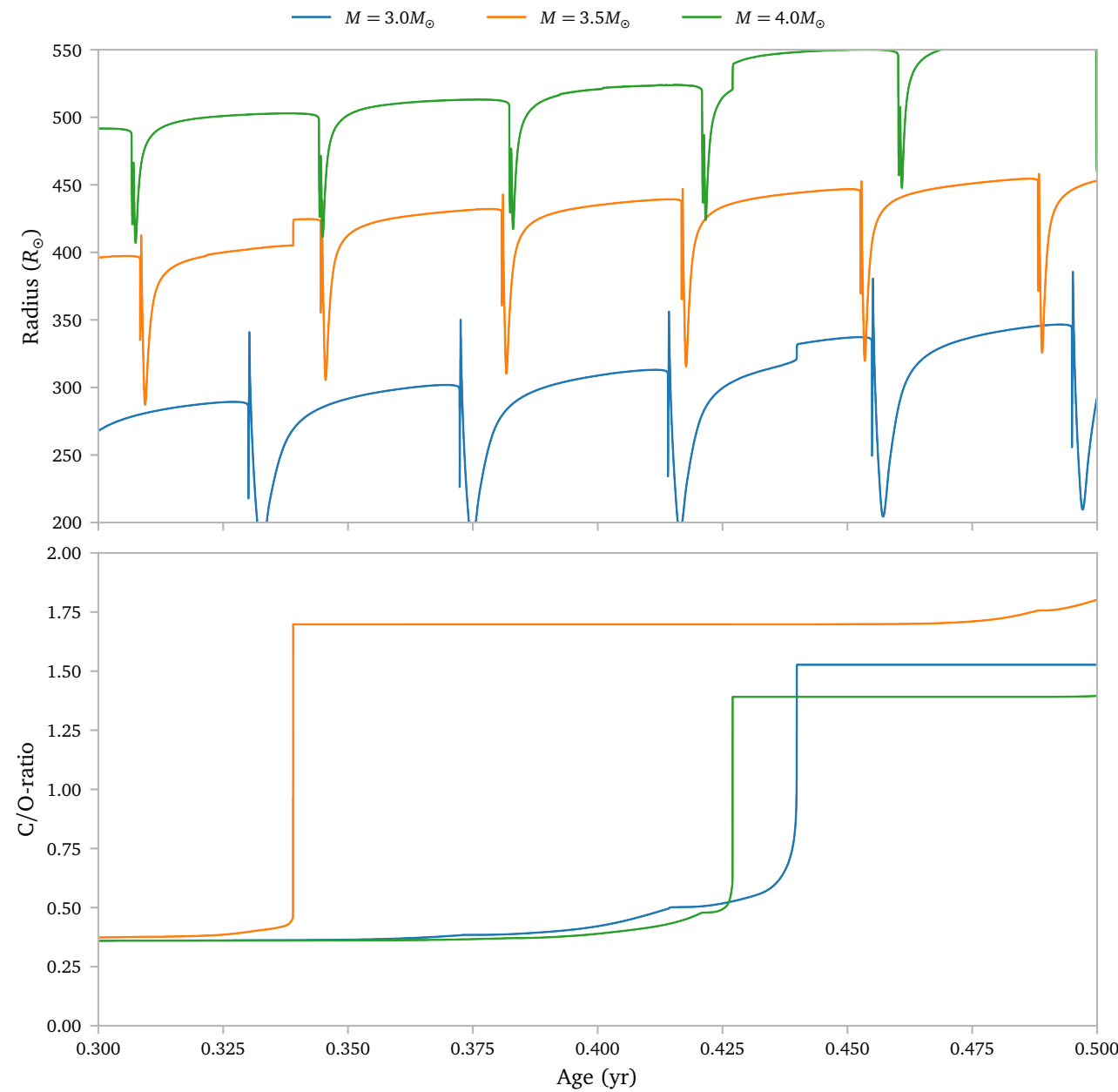
This figure shows the radius, C/O-ratio and the envelope fraction. Crosses indicate that the models crashed with terminal messages indicating that no opacity values can be found. Dots indicate that the models did not finish after 40 core days.



I tried my best to look into the issue, but surprisingly, I am not able to reproduce it on my own desktop. When I restart the model from the last photo, it is able to continue without any complaints.

I really do not know how this is possible. I checked that I modified the MESA source code to prevent the Fe-peak instability in both installations and this is indeed the case.

I am also pretty surprised by the sudden jumps to C/O-ratio for some of the models. This actually non-trivially changes the radius as well. I tried to show this in the plot below



The jumps also do not really align with where I would expect dredge up to occur.

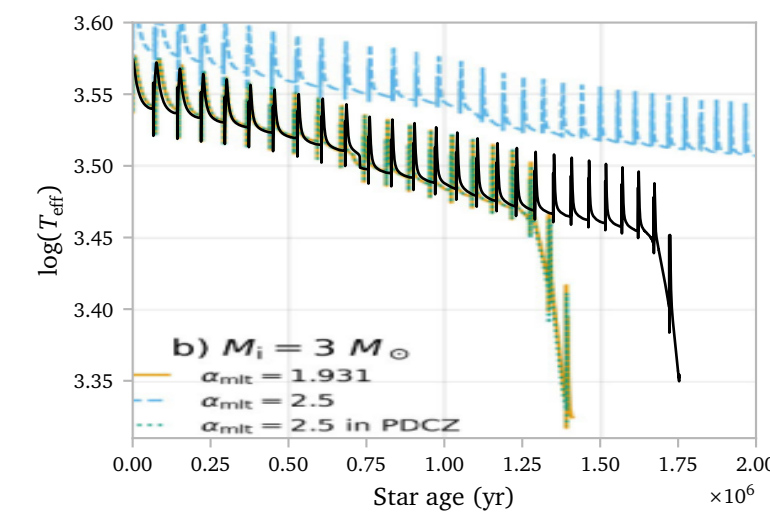
I tried to match the sudden increase with other changes in the star but I really couldn't find anything. The convection zones also do not change at the points where the ratio changes suddenly.

Unfortunately I was not able to get any profile data for the sudden jump because MESA was being really annoying and not allowing me to change the profile interval after a restart.

I was however able to reproduce the exact same jump after a restart on my local desktop, so unlike the crashes, this does not seem to change with the MESA version.

Rees et al. (2024) and Rees & Izzard (2024) do not mention these jumps, nor do they show clear figures of the models where I see these jumps. The only model where I see a somewhat similar situation is the model they show in Figure 1 and Figure 5 of Rees & Izzard (2024).

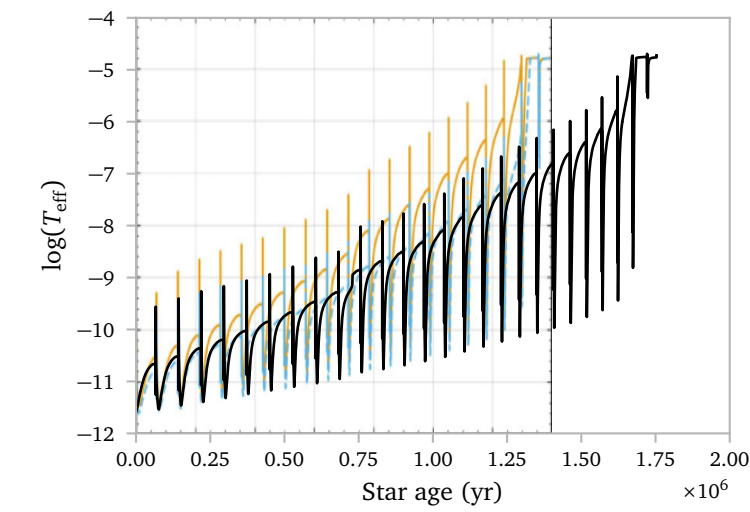
Unfortunately Rees et al. (2024) have not published the simulation data so I cannot accurately compare, but I can try to reproduce their Figure 1 and Figure 5.



Unfortunately, this is the best I could do. The image in Rees & Izzard (2024) is really pixelated.

We do see a similar drop at approximately the same time and effective temperature. This does not prove that this is a real physical thing but it does hint that Rees et al. (2024) saw similar jumps.

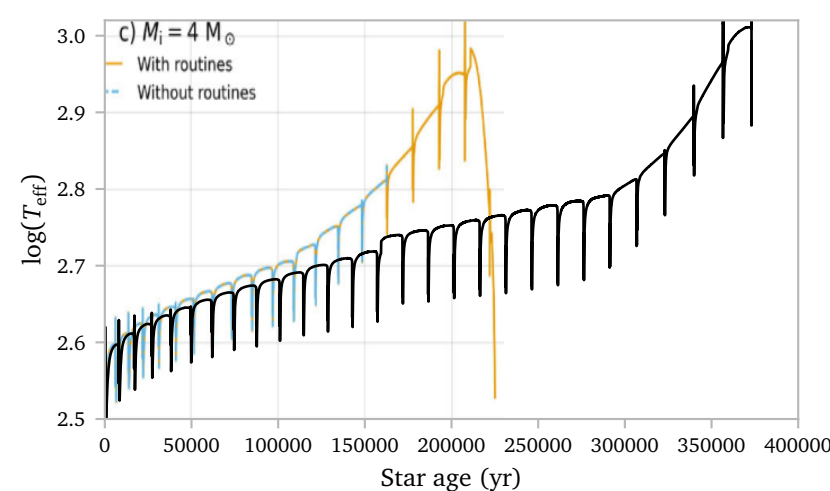
The differences towards the end of the evolution must be due to the modifications we made to the wind mass loss prescription, which now uses a constant wind velocity of 15 km/s.



Again, this is the best I could do. I think Rees et al. (2024) only computed a model with the Trabucchi et al. (2018) mass loss rate and used this model to infer the mass loss rate from a Vassiliadis & Wood (1993) wind mass loss prescription. Otherwise, the pulses definitely would not align perfectly. We can see however that our model has a lower mass loss rate.

The orange line is Trabucchi et al. (2018) mass loss while the blue lines are Vassiliadis & Wood (1993).

Below, I plot the radial evolution of the 4 M_sun star.



For larger masses, the evolution becomes drastically different and it is no longer possible to compare the evolution from Rees et al. (2024) with the evolution that we obtain using our modified Vassiliadis & Wood (1993) prescription.

c. 3.5 M_sun Star Grid

c.1 Model selection code

Looking at the first plot, I do think it is possible to run some models with a 3.5 M_sun single star.

In order to achieve this, I need to make sure the correct starting models are selected. I modified my grid-maker.py code to now look at the predicted Roche lobe radius during the evolution and pick a model based on that.²

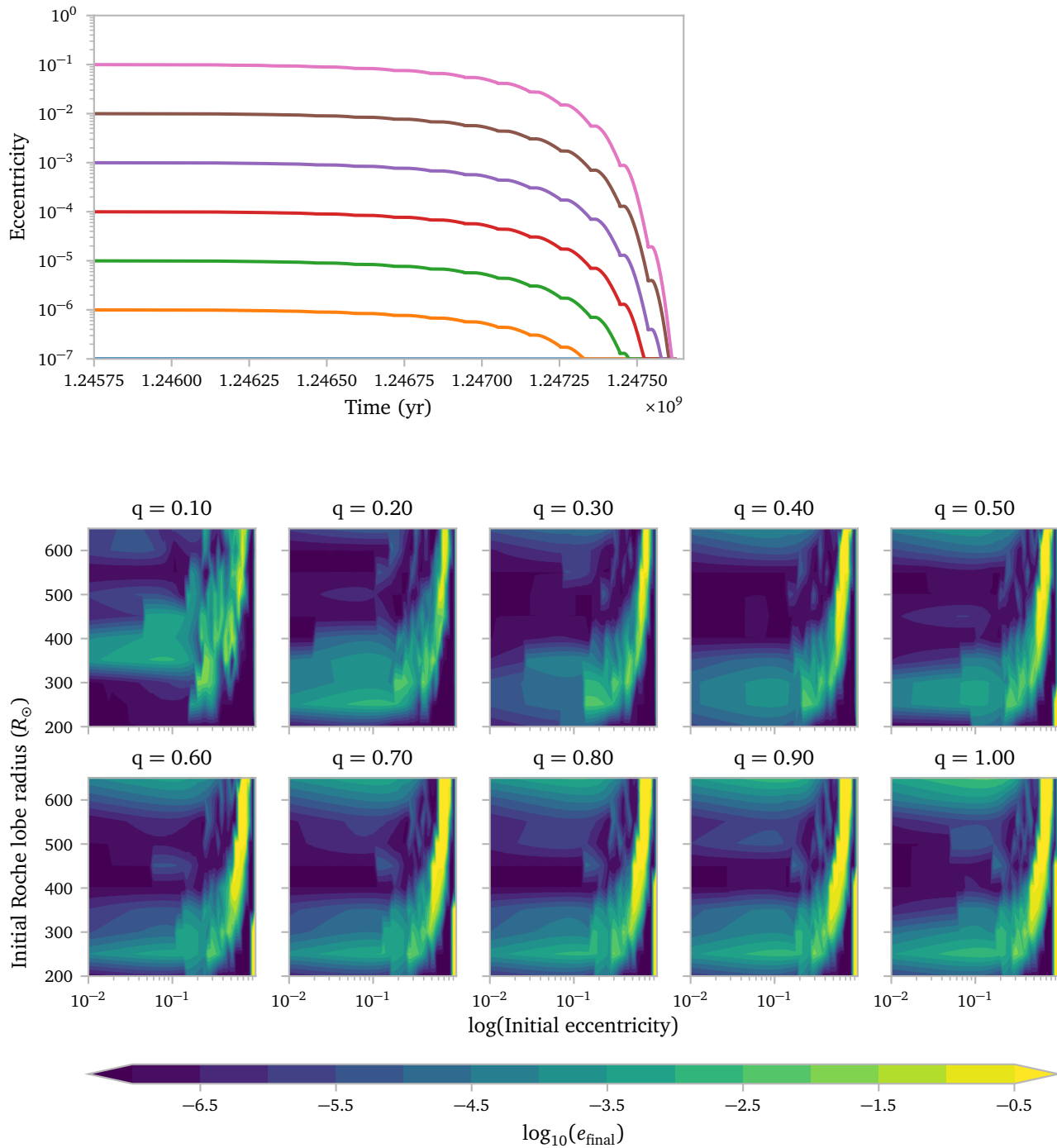
² The code can be seen in appendix B

c.2 Results

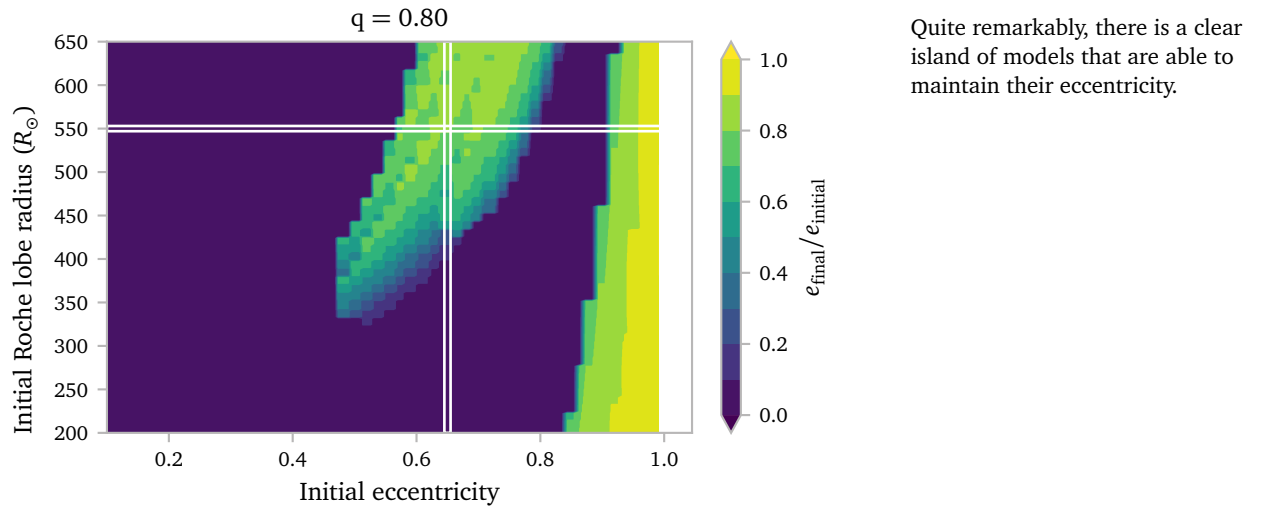
d. Non-zero Eccentricity

This is a bit unrelated to what I have been doing, but I was interested in seeing the efficiency of the [Hut \(1981\)](#) tidal prescription in circularizing the orbits before RLOF.

I am interested in this, not only because of the relevance to this project, but also because of the papers by [Parkosidis et al.](#) which predict that eccentricity during RLOF can significantly influence the final observed properties of binaries.

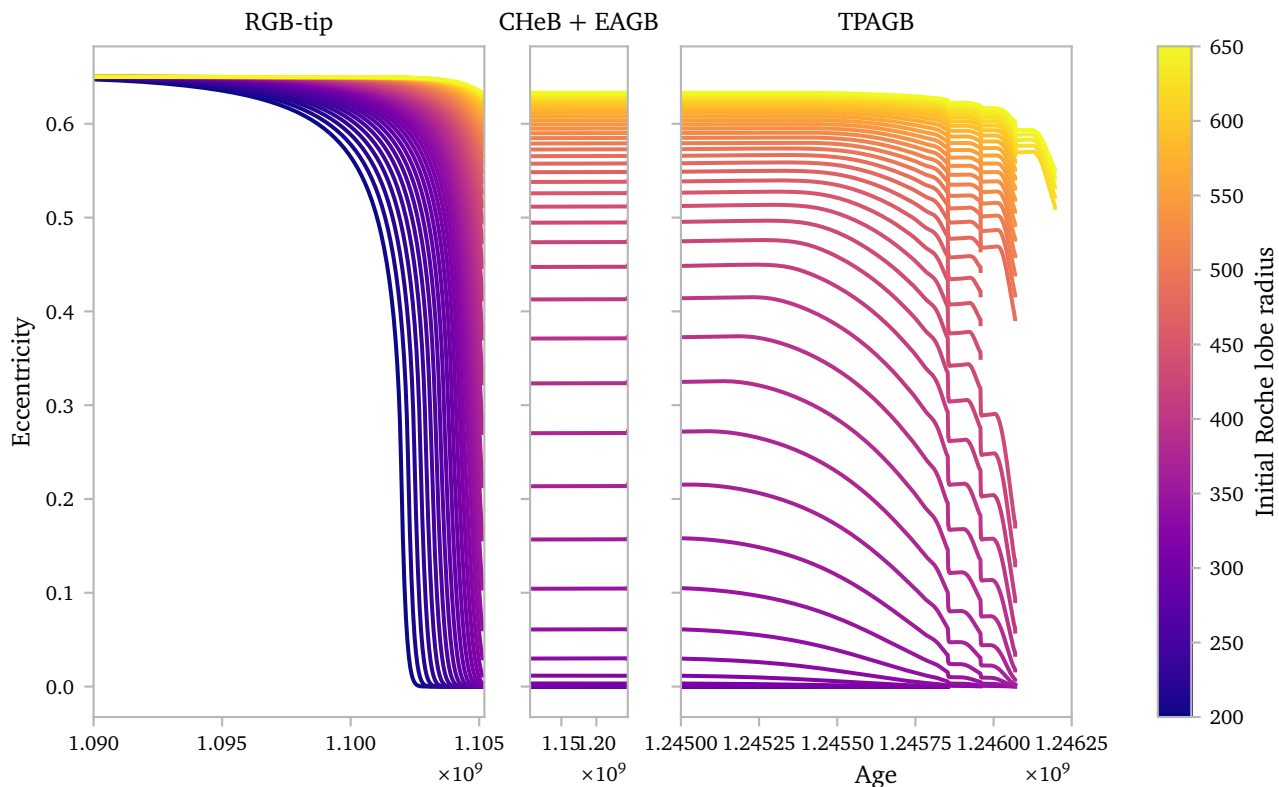


If we look more closely to models with a larger starting eccentricity, we obtain the following plot:



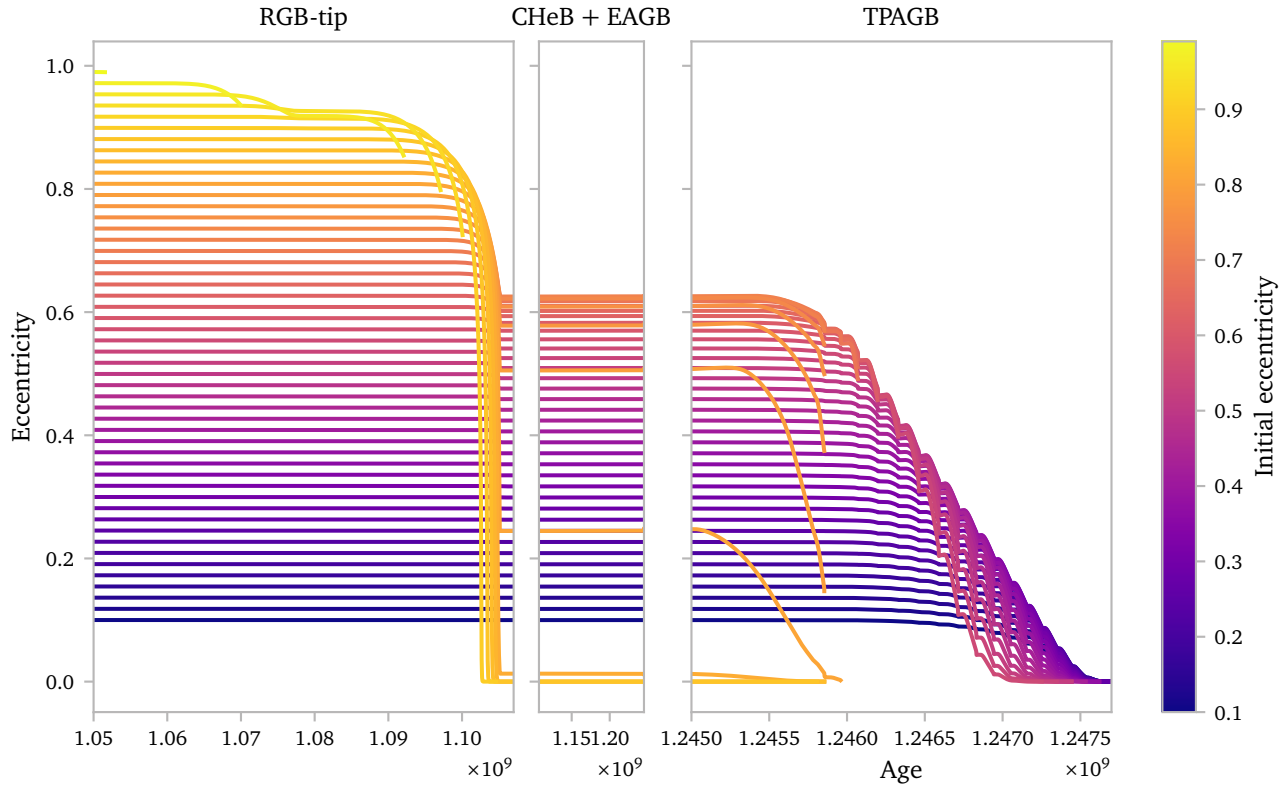
Quite remarkably, there is a clear island of models that are able to maintain their eccentricity.

I want to look at some slices to see how this is possible. First, the column:



Apparently, the RGB phase can already quite significantly circularise some of the shorter periods. Enough to make sure the eccentricity becomes very close to zero during the TPAGB phase.

Now the row:



Apparently some of the very eccentric orbits reach RLOF during the RGB tip, even for very large initial Roche lobe radii. These models will keep a large eccentricity. The models that just barely graze the RGB tip will circularize efficiently. Then, there are some models that reach RLOF pretty quickly during the TPAGB phase. These will also maintain some eccentricity. All other models circularize. Overall, a lot of the models are actually very circular when RLOF starts.

e. Outer Lobe Overflow

The paper from [Marchant et al. \(2021\)](#) is really interesting, and their prescription of outer lobe overflow is described in great detail with the aim to be easily implemented in MESA. They also provide code files of their modified mass transfer prescriptions.

I *think* it should be possible, but definitely not *easy* to implement their mass transfer prescription. This would probably be a pretty good step forward, as it naturally handles the large overflows we see.

Interestingly, [Marchant et al. \(2021\)](#) note smaller overflows when using their outer lobe prescription. They also find that not a lot of mass is actually lost through the outer lagrange points. This probably holds for our TPAGB stars as well.

References

Hut, P. (1981) *Tidal evolution in close binary systems.*. A&A99, 126.

Marchant, P., Pappas, K. M. W., Gallegos-Garcia, M., et al. (2021) *The role of mass transfer and common envelope evolution in the formation of merging binary black holes.* A&A650, A107.

Parkosidis, A., Toonen, S., Dosopoulou, F., & Laplace, E. (2026) *Rethinking mass transfer: A unified semianalytical framework for circular and eccentric binaries: I. Orbital evolution due to conservative mass transfer.* A&A706, A79.

Rees, N. R. & Izzard, R. G. (2024) *Evolving past instabilities on the thermally pulsing-(super)asymptotic giant branch.* MNRAS531, 4033.

Rees, N. R., Izzard, R. G., & Karakas, A. I. (2024) *Thermal pulses with MESA: resolving the third dredge-up.* MNRAS527, 9643.

Soberman, G. E., Phinney, E. S., & van den Heuvel, E. P. J. (1997) *Stability criteria for mass transfer in binary stellar evolution.*. A&A327, 620.

Trabucchi, M., Wood, P. R., Montalbán, J., et al. (2018) *Long-Period Variables in the Large Magellanic Cloud.* 514, 111.

Vassiliadis, E. & Wood, P. R. (1993) *Evolution of Low- and Intermediate-Mass Stars to the End of the Asymptotic Giant Branch with Mass Loss.* ApJ413, 641.

A Model saving code

```
subroutine save_model_on_radius_increase(id, s, ierr)

    integer, intent(in) :: id
    type(star_info), pointer :: s
    integer, intent(inout) :: ierr

    character(len=200) :: fname
    real(dp) :: R_now, R_last
    real(dp) :: R_max_since_last_save
    real(dp) :: R_max
    integer :: evol_phase
    logical :: needs_save

    needs_save = s% lxtra(lx_needs_save)
    evol_phase = s%x_integer_ctrl(1)

    ! the subroutine uses the current stellar radius
    ! and the last saved stellar radius to determine
    ! if a model should be saved.
    R_now = s% photosphere_R
    R_last = s% xtra(x_last_saved_R)
    R_max_since_last_save = s%
    ~ xtra(x_max_R_since_last_save)

    ! if true: save a model

    ! this criterion produces many models,
    ! but using this routine is only required once
    ! for every mass star!

    if (((R_now - R_last > 10d0) .or. (R_now > 1.25d0 *
    ~ R_last)) .and. (evol_phase > 1) .and. (evol_phase
    ~ /= 5)) then
        write(fname, fmt="(a7,f0.3,a7,i0,a4)") 'models/',
            ~ R_now, 'Rsun_TP', s% ixtra(ix_TP_count),
            ~ ',mod'

        call star_write_model(id, fname, ierr)

        s% need_to_update_history_now = .true.
        s% need_to_save_profiles_now = .true.

        open(1, file="last_saved_R.dat",
            ~ status="replace")
        write(1,*) R_now
        close(1)

        if (ierr /= 0) then
            write(*,*) 'error writing model'
            ierr = 0
        end if

        s% xtra(x_last_saved_R) = R_now
        s% xtra(x_max_R_since_last_save) = R_now
    end if

    if (evol_phase == 5) then
        ! in TPAGB phase

        if ((s%lxtra(lx_in_LHe_peak) .eqv. .true.) .and.
            ~ (needs_save .eqv. .true.)) then
            write(fname, fmt="(a7,f0.3,a7,i0,a4)")
                ~ 'models/', R_max_since_last_save,
                ~ 'Rsun_TP', s% ixtra(ix_TP_count), ',mod'

            call star_write_model(id, fname, ierr)

            s% need_to_update_history_now = .true.

            open(1, file="last_saved_R.dat",
                ~ status="replace")
            write(1,*) R_now
            close(1)

            if (ierr /= 0) then
                write(*,*) 'error writing model'
                ierr = 0
            end if

            s% xtra(x_last_saved_R) = R_now
            s% lxtra(lx_needs_save) = .false.
        end if

        if (R_now - R_max_since_last_save > 0d0) then
            s% xtra(x_max_R_since_last_save) = R_now
        end if

        if (s%lxtra(lx_in_LHe_peak) .eqv. .false.) then
            s% lxtra(lx_needs_save) = .true.
        end if
    end if
end subroutine save_model_on_radius_increase
```

B Model Selection Code

```
models_dict = get_model_dict()
tpagb_age = _get_tpagb_age()
ref_eagb_age =
    ~ mr.MesaData(f"{ref_dir}/eagb.data").star_age[-1]

Star = read_stellar_models(standard_dir)[0]

for R in Rs:

    for q in qs:
        a_init = inv_roche_lobe(R, q)
        [Star, Options, q_init, a_init, e_init, Bins] =
            ~ call_evolution(
                Star, q, a_init, simple_only=True
            )

        R_star = Star.R
        ages_star = Star.age
        bin = Bins[0]

        q_evolve = bin.m2 / bin.m1
        RL = roche_lobe(1 / q_evolve) * bin.a

        # interpolate stellar radius onto binary ages
        R_interp = np.interp(bin.age, ages_star, R_star)

        # find first near-contact point
        inds = np.where(R_interp > 0.9 * RL)[0]

        if len(inds) == 0:
            print("system never approaches RLOF ->
                ~ skipping")
            continue

        contact_index = inds[0]
        contact_age = bin.age[contact_index]

        candidate = None

        for key in list(models_dict.keys())[:-1]:

            model = models_dict[key]

            model_path =
                ~ f"{grid_dir}/models/{model['name']}"
            mod_data = mr.MesaData(model_path)

            model_age = _find_initial_age(mod_data) +
                ~ tpagb_age

            if model_age < contact_age:
                candidate = model
                break

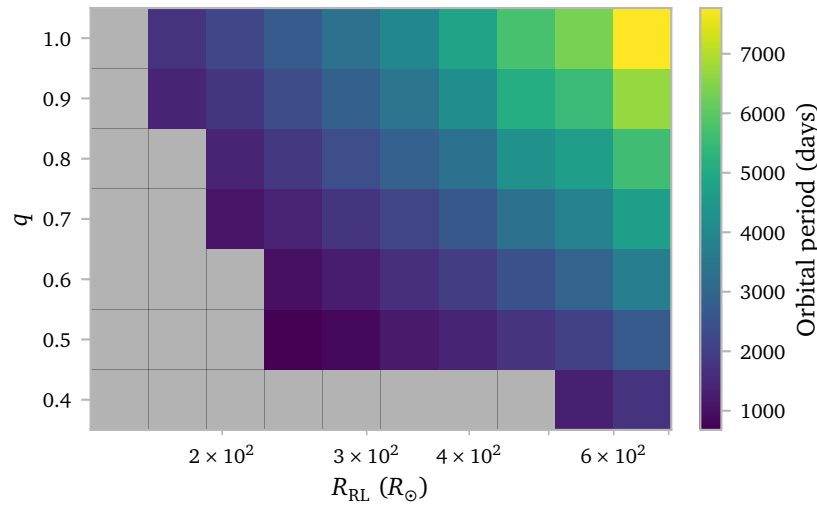
        if candidate is None:
            print("no suitable stellar model before
                ~ contact -> skipping")
            continue

        model = candidate
        mass = model["M"]
        model_path = f"{grid_dir}/models/{model["name"]}"
        mod_data = mr.MesaData(model_path)
        model_age = _find_initial_age(mod_data) +
            ~ tpagb_age

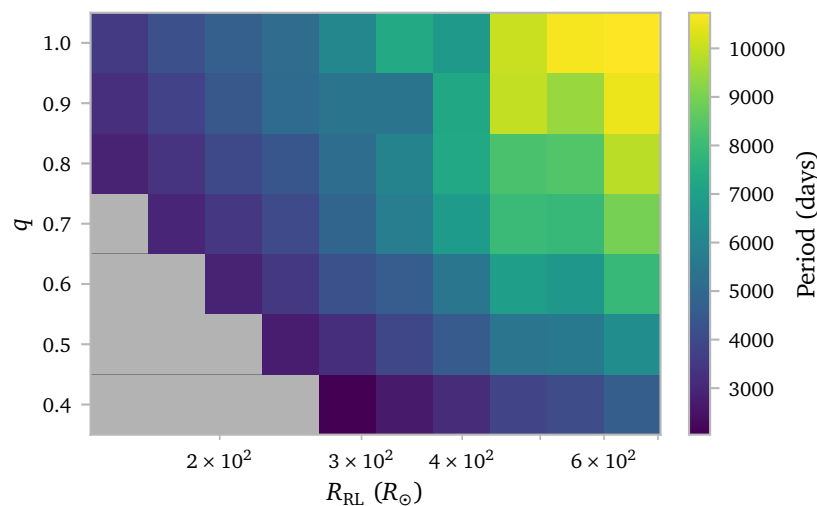
        index = np.argwhere(bin.age > model_age)[0][0] -
            ~ 1
        index_star = np.argwhere(Star.age >
            ~ model_age)[0][0] - 1
        q_evolve = bin.m1 / bin.m2
        m1 = bin.m1[index]
        m2 = bin.m2[index]
        a = bin.a[index]
        omega = bin.spin1[index]
        varcontrol = Star.varcontrol[index_star]

        print(f"\nR = {R:.0f}, q = {q:.2f}")
        run_subdir = f"runs/R{R:05.2f}_q{q:.3f}"
        run_dir = f"{grid_dir}/{run_subdir}"
        print(f"copying reference binary to
            ~ {run_subdir}...")
        shutil.copytree(f"{grid_dir}/reference-binary",
            ~ run_dir)
        inlist_dir = f"{run_dir}/inlist_project"
        inlist_star = f"{run_dir}/inlist1"
        print(f"copying {model["name"]} to
            ~ {run_subdir}...")
        shutil.copyfile(model_path,
            ~ f"{run_dir}/start.mod")
        print(f"changing inlist...")
        change_inlist_bin(a, m1, m2, inlist_bin)
        change_inlist_star(omega, varcontrol,
            ~ inlist_star)
```

C Additional figures



This plot shows the periods for models with $\delta = 0.2$, $\gamma = 1.25$ and the radiation corrected mass accretion rate. Return to Section a.



This plot shows the periods for models with fully conservative mass transfer. Return to Section a.